

Practical Assignment: Denials Command Center

AQSoft (AQKODE Healthcare Solutions) | Full-Stack Developer | Time box: 72 hours

Why this assignment

We do not write detailed specifications. Our developers sit with the people who do the work, find the real problem, and ship something they can trust. This assignment tests exactly that: can you find the problem hidden in messy real-world data, use AI well to solve it, and build a working product quickly without being told how.

It is deliberately hard and deliberately open. You will not finish everything in 72 hours. We want to see what you choose to build first, how correct it is, and how clearly you explain your decisions.

The situation

AQKODE handles coding, billing and denials for **Gulfview Physician Partners**, a group of hospital and nursing-facility doctors in Florida. They bill four insurance payers. When a payer refuses to pay a claim (a "denial"), our denials team works it from an Excel log and the payer's electronic remittance files (ERA / X12 835).

Every Monday the practice manager asks three questions that nobody can answer quickly or reliably:

1. How much money is stuck in denials right now, and how much of it can we still recover?
2. Why are we being denied, and which of these denials should never have left the building?
3. What should each person on my team work on today?

Your job is to build the product that answers these three questions.

What you get (data pack)

File	What it is
README.txt	Start here: file list and column definitions
claims_export.csv	Every claim line we billed Jan to Aug 2026 (patient, payer, dates, provider, facility, CPT, modifiers, ICD-10 codes, charge, coder, whether pre-bill review happened)
remits/*.835	Payer remittance files in X12 835 format, exactly as received from the clearinghouse
denials_worklog.xlsx	The denials team's manual Excel tracker
payer_policies/*.md	Excerpts of payer policies (reimbursement, coding, authorization, provider enrollment)
payer_rules.csv	Timely filing, appeal and corrected-claim windows by payer
carc_rarc_reference.csv	Standard denial (CARC) and remark (RARC) code descriptions
claim_adjustment_group_codes.csv	Meaning of the adjustment group codes (CO, PR, OA, PI)
labeled_denials_sample.csv	40 denials already reviewed by an expert (root cause, owning team, preventable before billing) for evaluating your AI

All data is fictional, but it was produced the way real systems produce it. Do not assume it is clean, complete or consistent. Treat "today" as 30 September 2026. Design as if it were real patient health information.

What to build

These are outcomes, not a specification. How you design it is up to you; justify your choices.

A. Problem memo (max 1 page)

What are the real problems you found, both business and data? What did you build first and why? What would you build next? Include the key numbers that answer the manager's three questions.

B. Ingestion and reconciliation

- Load all inputs and produce one trustworthy record per claim, with its complete payment and denial history.
- Re-running on the same files must not change any result.
- Anything that cannot be matched or understood goes to an exceptions view with a reason. Nothing is silently dropped.
- Show how your numbers reconcile (for example: what the payer files say was paid vs what your system shows).

C. AI denial analysis

- For every open denial: root-cause category, owning team, whether it was preventable before billing, and the recommended next action.
- For appealable denials: a draft appeal or correction note that cites the exact payer policy section it relies on.
- Show a confidence level; send low-confidence results to a human review queue.
- The system must still work (degraded) if the AI service is unavailable.
- Evaluate your AI against labeled_denials_sample.csv. Report accuracy and where it fails, and why.
- Treat all text coming from files as untrusted input.

D. Worklist application (React + TypeScript)

- Two roles: **Denials Specialist** (sees own queue, updates status and notes) and **Manager** (sees everything, reassigns work).
- A queue prioritised by what actually matters. You decide the logic and explain it.
- A claim detail page showing the full timeline, from billed to paid or denied, plus every action taken.
- Every change is audit-logged: who, what, when, before and after.

E. Manager analytics and prevention

- Money at risk: recoverable vs already lost, trends, and breakdowns by payer, reason, provider, coder and facility.
- A prevention view: which checks, applied before a claim is sent, would have stopped the most denials and money.
- Export those checks as machine-readable rules (JSON) that our pre-bill review team could apply to future claims.

Technical guidelines

- Backend in C# / .NET. Frontend in React + TypeScript. Database SQL Server or PostgreSQL. Python is fine for data or AI components.
- Any LLM (OpenAI, Anthropic, Gemini or a local model). AI coding tools are encouraged; you must understand and be able to explain every line.

- One-command start (docker compose preferred). Never commit API keys; tell us which environment variables to set.
- Automated tests for the parts where a mistake would cost money.

What to submit

- Private GitHub repository shared with the GitHub account named in our email.
- README: how to run it, architecture, key decisions and trade-offs, what you did not finish and why.
- The problem memo (A).
- AI evaluation report (C).
- AI usage log (1 page): where AI helped you most, where it was wrong, and how you caught it.
- A 5 to 8 minute screen recording walking through the product.
- Honest total hours spent.

Time and the walkthrough

You have **72 hours** from receiving this. Email your submission (repository link, memo, reports and recording link) to the address in our email before the deadline stated there.

Correct and working beats complete: we would rather see three parts done right than five done loosely.

Shortlisted submissions get a 60 to 75 minute walkthrough with our team: a demo, a deep dive into your code and decisions, and a **live round** where we hand you a new remittance file and a change request, which you implement while sharing your screen (any tools allowed).

How we will judge

Area	What we look for
Correctness	Do your numbers match what the data really says?
Problem finding	Did you find what actually drives the losses, beyond what we asked?
AI quality and safety	Grounded, evaluated, honest about confidence, hard to fool
Product	Would the denials team and the manager actually use it on Monday?
Engineering	Structure, tests, idempotency, audit trail, security
Speed and communication	How much working value you delivered in the time, and how clearly you explain it

Questions are welcome, the way you would ask an operations manager, but we will not tell you how to solve it. Good luck.